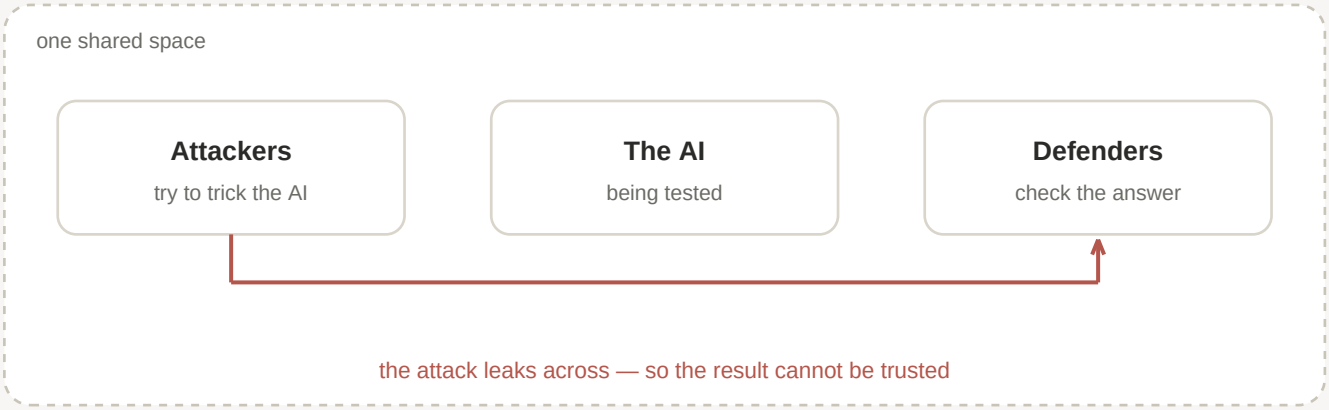
sealed

Safe, trustworthy AI security testing — the complete guide, in plain words.

To check whether an AI is safe, one team tries to trick it (the attackers) while another team defends it (the defenders), at the same time. The problem: if they share one space, they can peek at each other and the AI gets confused between tests, so the result cannot be trusted. sealed keeps everyone in separate rooms, routes everything through one trusted go-between, gives the AI a fresh memory for each test, and keeps a record no one can secretly change — then issues a signed proof that the test was sound.

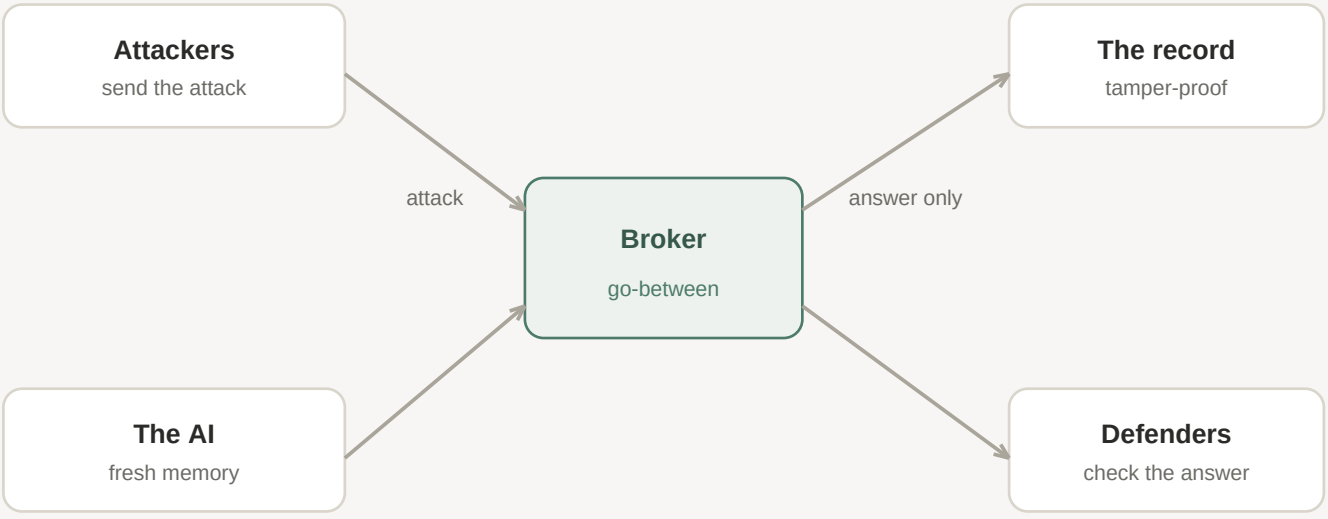
1. The problem

Attackers, the AI, and defenders all share one space. The attack leaks over to the defenders (and the AI remembers things between tests). The “pass” means nothing.



2. The solution

Each team gets its own locked room. Everything passes through one trusted go-between (the broker). The defenders get only the AI’s answer — never the raw attack — until the test ends. Every step is written to a tamper-proof record.



3. Everything it can do

Feature	In simple words

Separate rooms	Each team is locked away and cannot reach, see, or read any other team (separate networks, blocked dangerous actions, removed powers, read-only files, resource limits).
The go-between	All messages pass through one trusted middleman — no team ever talks to another directly.
Access passes	Each team can only do what its role allows; anything else is blocked and logged.
Hiding the attack	The defenders cannot see the raw attack until the test ends — so they can't cheat.
Fresh AI each test	The AI's memory is wiped between tests, so a trick in one test can't leak into the next.
Tamper-proof record	Every action is written in a chained, digitally-signed log; any edit, delete, or re-order is caught.
Security watcher	A one-way watcher records any rule-breaking. No team can read it.
Proof certificate	After a test, sealed issues a signed certificate that it ran in a sound, isolated environment. Anyone can verify it.
Findings report	A one-page PDF of the results, stamped with the signed proof — ready to share.
Live tamper check	A button that secretly edits a record and shows the checker catching it.
Before / after demo	Shows the leak happening in a naive setup, then the same attack fully contained.
Real benchmark attacks	Replays the real AdvBench benchmark — recognized real-world attack prompts.
Real AI model (optional)	Run against a real model (Ollama) instead of the built-in stand-in.
Plain-language website	An overview page, a 1-minute animated walkthrough, and a live console — all in simple words.
Auto-tested	Every change is automatically tested on a real Linux machine (CI) and must pass.
Honest about limits	A threat model that states plainly what it does and does not protect.

4. What it proves (the six goals)

Goal	How it is met
Stay fast while safe	The extra security doesn't slow the test to a crawl; the cost is measured.
Stop information leaking	Teams can't reach or watch each other; the attack is hidden from defenders.
Keep an honest record	A tamper-proof log where any change is caught and pointed to.
Run anywhere normal	Works on an ordinary Linux server with standard tools — no special hardware.

Handle AI-specific risks	A fresh AI memory per test, so one test can’t poison the next.
Be honest about gaps	The limits are written down as clearly as the protections.

5. How to test it — step by step

There are two easy ways. Pick A if you have a Linux machine with Docker; pick B if you only have Python. Then try the website (C).

A) Full test on a Linux machine (Docker or Podman)

Run this	What you should see
`cd sealed && cp .env.example .env`	Prepares the settings file.
`make up`	Builds and starts the 6 parts of the system.
`make ps`	All 6 show “healthy”.
`pip install -r tests/requirements.txt`	One-time: installs the test tool.
`make test`	All checks pass (isolation, record, contamination, access, speed, proof).
`make demo-naive`	The BEFORE: shows the leak (leaked: true) and the defender reading the attack.
`make demo-secure`	The AFTER: same attack contained (leaked: false), plus the record of every step.
`make verify-audit`	The record checks out: intact.
`make demo-tamper`	Secretly edits a copy of the record — the checker flags the exact changed entry.
`make fetch-datasets`	Downloads the real AdvBench benchmark attacks (optional).
`make up-ollama`	Runs against a real AI model instead of the stand-in (optional, needs internet).
`make down`	Stops everything.

B) Quick check with only Python (no Docker)

Run this	What you should see
`pip install -r sealed/tools/requirements.txt`	Installs what the quick check needs.
`python sealed/tools/local_e2e.py`	Runs everything as local programs and prints “17/17 checks passed”.

This proves the logic (the eval loop, fresh memory, access control, the record, and the signed proof). The container-only parts (the separate-rooms isolation) need path A or the automatic online tests.

C) Try the website

- Start it: `cd web, then python -m http.server 8080`, and open http://localhost:8080 in a browser.
- Overview page: the problem, the solution, and the plain-English limits.
- Walkthrough: a 1-minute animated story (arrow keys to move, space to pause).
- Console: with the system running (path A), press Connect, then: Run a test → Get the proof → Check the proof is genuine → Download findings report (PDF) → Show tamper detection.

6. How to know it worked

- make up: all 6 parts show “healthy”.
- make test and python local_e2e: every check passes (17/17).
- make demo-naive shows a leak; make demo-secure shows it contained — that contrast is the point.
- make verify-audit says intact; make demo-tamper catches the edit.
- In the console: the proof is “genuine”, the report downloads, and tamper detection is “caught”.
- Online: the project’s automatic tests (CI) show a green tick on GitHub.

7. Honest limits

sealed stops the big risks (peeking, contamination, tampering, unauthorized requests) and reduces subtler ones (timing and shared-resource clues). It does not claim to stop deep chip-level leaks on shared processors, a hacked server, or a dishonest go-between. Being clear about this is what makes the rest trustworthy.

8. Where it fits

Other AI security tools (Garak, PyRIT, Giskard, and similar) are good at generating attacks and finding weaknesses. sealed is different and complementary: it doesn’t generate cleverer attacks — it guarantees the test itself was sound, and gives you a proof of it. In short: those tools find weaknesses; sealed makes the finding trustworthy.

Repository: github.com/ChethanValukuru/sealed-manali-